

# Dokumentacja projektu

---

## Gra 2D typu Flappy Bird w języku C++

**Projekt:** Gra 2D Flappy Bird w C++/Qt 6

**Przedmiot:** Programowanie II - C++ obiektowy

**Prowadzacy:** dr inż. Marcin Sobota

**Uczelnia:** Politechnika Śląska, Wydział RMS

**Semestr:** 2025/2026, sem. 2

**Grupa:** Olaf Rysiewicz, Dawid Skatula, Szymon Panek

## Autorzy

| Imię i Nazwisko | Rola w projekcie                                     |
|-----------------|------------------------------------------------------|
| Olaf Rysiewicz  | Klasa Bird, fizyka ptaka, rysowanie QPainter         |
| Dawid Skatula   | Klasa Pipe, PipeFactory (wzorzec Factory), walidacja |
| Szymon Panek    | GameWindow, petla gry, kolizje, UI, dokumentacja     |

## 1. Opis zrealizowanego projektu

Zrealizowaliśmy grę 2D inspirowaną Flappy Bird. Aplikacja okienkowa napisana w C++ z użyciem Qt 6. Gracz steruje ptakiem, który ciągle opada pod wpływem grawitacji. Naciśnięciem spacji lub kliknięciem myszki gracz daje ptakowi impuls w górę. Celem jest omijanie pionowych rur i zdobywanie punktów za każdą ominiętą parę.

### Funkcjonalności

- \* Ekran startowy z tytułem, instrukcją i nazwami autorów
- \* Fizyka ptaka: grawitacja z ograniczeniem prędkości, skok po spacji/kliknięciu
- \* Losowe przeszkody: rury z losową wysokością przerwy, przesuwające się w lewo
- \* Detekcja kolizji: z rurą, ziemią i górną krawędzią ekranu
- \* System punktacji: +1 za każdą ominiętą parę rur
- \* Zapis rekordu w trakcie sesji
- \* Ekran końca gry z wynikiem, rekordem i opcją restartu
- \* Grafika generowana kodem - zero zewnętrznych plików

### Stos technologiczny

| Element | Szczegóły                      |
|---------|--------------------------------|
| Język   | C++17                          |
| GUI     | Qt 6 (Core, Widgets)           |
| Grafika | QPainter (rysowanie wektorowe) |
| Build   | CMake 3.16+, qt_add_executable |

|     |                    |
|-----|--------------------|
| IDE | Visual Studio Code |
|-----|--------------------|

## 2. Architektura i elementy OOP

### Hierarchia klas

|                |                                  |
|----------------|----------------------------------|
| QWidget        |                                  |
| +-- GameWindow | głowne okno, zarządza stanem gry |
| GameObject     | klasa abstrakcyjna (bazowa)      |
| +-- Bird       | ptak gracza                      |
| +-- Pipe       | para rur (przeszkoda)            |
| +-- Ground     | podłoże                          |
| PipeFactory    | wzorzec Factory, tworzy rury     |

### Dziedziczenie i polimorfizm

Klasa GameObject definiuje wspólny interfejs (draw, update - czysto wirtualne). Klasy Bird, Pipe i Ground dziedziczą po GameObject i nadpisują obie metody. Dzięki temu GameWindow obsługuje wszystkie obiekty jednakowo przez wskaźnik:

```
for (Pipe* pipe : pipes) {  
    pipe->draw(painter);    // wywołuje Pipe::draw()  
    pipe->update();          // każda rura wie jak się poruszać  
}
```

### Konstruktory i destruktory

Każda klasa ma konstruktor domyślny, parametryczny i destruktor. Destruktor ~GameObject() jest virtual - konieczne przy polimorfizmie, bo bez tego delete na wskaźniku bazowym nie wywołałby destruktora klasy pochodnej.

### Obsługa wyjątków

Pipe::Pipe() waliduje parametry i rzuca wyjątki. PipeFactory łapie i stosuje bezpieczne wartości domyślne:

```
// Pipe::Pipe()  
if (gapHeight < 90)  
    throw std::invalid_argument("Przerwa za mała! Minimum 90px.");  
  
// PipeFactory::createPipe()  
try { return new Pipe(x, gapCenter, gapHeight, screenHeight); }  
catch (const std::invalid_argument& e) {  
    return new Pipe(x, screenHeight/2, 170, screenHeight); // fallback  
}
```

### Wzorzec Factory Method

PipeFactory odpowiada za tworzenie rur (losowanie pozycji, walidacja, predkosc). GameWindow tylko wywołuje pipeFactory->createPipe() - nie zna szczegółów tworzenia.

### 3. Instrukcja uzytkownika

#### Sterowanie

| Akcja              | Klawiatura               | Mysz                |
|--------------------|--------------------------|---------------------|
| Start gry          | Spacja                   | Lewy przycisk myszy |
| Skok ptaka         | Spacja lub strzałka gore | Lewy przycisk myszy |
| Restart po smierci | Spacja                   | Lewy przycisk myszy |
| Zamkniecie gry     | Escape                   | Przycisk X okna     |

### 4. Ciekawostki z realizacji

#### Rysowanie bez assetow

Dziob ptaka to QPolygon z trzema punktami. Efekt 3D rury to jasniejszy pasek po lewej stronie. Chmury to nakladajace sie elipsy. Gradient nieba robi QLinearGradient. Mozna zrobic duzo bez jednego pliku PNG.

### 5. Podsumowanie - odniesienie do zalozen wstepnych

#### Zrealizowane zalozenia

| Zalozenie                   | Uwagi                                 |
|-----------------------------|---------------------------------------|
| Dziedziczenie + polimorfizm | GameObject -> Bird, Pipe, Ground      |
| Konstruktory i destruktory  | Kazda klasa; ~GameObject() wirtualny  |
| Obsluga wyjatkow            | Pipe::Pipe() rzuca, PipeFactory lapie |
| Wzorzec projektowy          | Factory Method w PipeFactory          |
| Walidacja danych            | Parametry rur, poziom trudnosci       |
| Grafika bez assetow         | Tylko QPainter                        |

#### Czego sie nauczylismy

- \* Zarzadzanie pamiecia w C++ - new/delete, wektory wskaznikow
- \* Polimorfizm w praktyce - faktyczna technika upraszczajaca petla gry
- \* Hitboxy vs grafika - dwie oddzielne rzeczy; dobry hitbox = dobry gameplay

### 6. Mozliwosci rozbudowy projektu

| Pomysl                     | Opis                                                      |
|----------------------------|-----------------------------------------------------------|
| Efekty dzwiekowe           | Qt Multimedia (QSoundEffect) - ding przy punkcie, crash   |
| Animacja ptaka             | Kilka klatek rysowanych naprzemiennie co 5 klatek gry     |
| Zapis rekordu do pliku     | QFile + QDataStream - rekord przetrwa zamkniecie programu |
| Tryb multiplayer (lokalny) | Drugi ptak sterowany klawiszami                           |
| Poziomy / motywy           | Rozne tla i kolory rur per poziom                         |